UVM
UNIVERSIDAD VIÑA DEL MAR

Laboratorio 5

Lenguaje máquina ensamblador

Prueba el siguiente código en
https://onecompiler.com/assembly

```
section .data
	hello:     db 'Hello, World!',10    ; 'Hello, World!' plus a
linefeed character
	helloLen:  equ $-hello              ; Length of the 'Hello world!'
string

section .text
	global _start

_start:
	mov eax,4            ; The system call for write (sys_write)
	mov ebx,1            ; File descriptor 1 - standard output
	mov ecx,hello        ; Put the offset of hello in ecx
	mov edx,helloLen     ; helloLen is a constant, so we don't need
to say
                     ;  mov edx,[helloLen] to get it's actual value
	int 80h              ; Call the kernel
	mov eax,1            ; The system call for exit (sys_exit)
	mov ebx,0            ; Exit with return "code" of 0 (no error)
	int 80h;
```

Esto es un código en ensamblador, assembly, para Linux en arquitectura X86 que imprimie un Hello, World!

1. Sección de datos

```
section .data
hello:     db 'Hello, World!',10
helloLen:  equ $-hello
```

- section .data
  Aquí se definen **variables o datos**.
- hello:
  Es una etiqueta (como una variable) que apunta al texto.
- db 'Hello, World!',10
  - db = *define byte*
  - Guarda el texto en memoria
  - 10 = salto de línea (ASCII)
- helloLen: equ $-hello
  - equ define una constante
  - $ = posición actual en memoria
  - $ - hello = calcula el **largo del texto**

2. Sección de código

```
section .text
global _start
```

- section .text
  Aquí va el **código ejecutable**
- global _start
  Define el punto de inicio del programa (como main en C/Java)

3. Inicio del programa

```
_start:
```

Aquí comienza la ejecución.

4. Llamada al sistema para escribir

```
mov eax,4
mov ebx,1
mov ecx,hello
mov edx,helloLen
int 80h
```

Esto es una **llamada al sistema (syscall)** en Linux:

- mov eax,4 → código de syscall **write**
- mov ebx,1 → descriptor 1 = pantalla (stdout)
- mov ecx,hello → dirección del texto
- mov edx,helloLen → tamaño del texto
- int 80h → interrupción para llamar al kernel

Equivalente conceptual en alto nivel:

```
write(1, "Hello, World!", 14);
```

5. Salida del programa

```
mov eax,1
mov ebx,0
int 80h
```

- eax = 1 → syscall **exit**
- ebx = 0 → código de salida (sin error)
- int 80h → ejecuta

Equivalente:

```
exit(0);
```

Resumen

Este programa:

1. Guarda el texto "Hello, World!"
2. Calcula su tamaño
3. Llama al sistema operativo para imprimirlo
4. Termina el programa

Actividad

Crea un programa que permite sumar y restar dos números asignados por código